SIMATS ENGINEERING

Department of Computer Science & Engineering

ASSESSMENT TOOL 1- ANALYTICAL PROBLEM SOLVING

Course Code:	CSA1223	Course Title:	Computer Architecture
CO Assessed:	CO2	Assessment:	ASSESSMENT TOOL1- ANALYTICAL PROBLEM SOLVING
Weightage:	45%	Total Marks:	100
CO2 Statement:	Apply integer and floating-point arithmetic algorithms including Booth's multiplication, restoring/non-restoring division, and IEEE 754 standard operations to solve fixed-point and floating-point computational problems. (BL3)		
Student Name:	Poorvaja .B	Reg.No.:	192473008
Department:	B.E-CSE (DS)	Date:	

75%

ANNEXURE A

1. A processor performs arithmetic operations on signed 8-bit integers using 2's complement representation. Consider two numbers: +45 and −23. Analyze in detail how the system performs both addition and subtraction, including binary conversion, alignment of sign bits, and intermediate steps. Determine whether overflow occurs, and explain how the processor detects overflow and interprets the final result.

2. A high-performance computing system replaces a ripple carry adder with a 4-bit carry look-ahead adder (CLA) to reduce computation delay. Given two binary inputs, analyze how generate and propagate signals are formed and how carries are computed in advance. Compare the time delay of CLA with ripple carry addition and justify, with reasoning, why CLA improves speed in modern processors.

3. In a signed multiplication operation, a processor uses Booth's algorithm to multiply two numbers, −6 and +5. Analyze the algorithm step-by-step, including the role of the accumulator, multiplier, and Booth bit, along with arithmetic shifts. Explain how the algorithm efficiently handles signed numbers and reduces the number of addition/subtraction operations compared to traditional multiplication.

4. A processor is required to perform division of two binary numbers (e.g., 13 ÷ 3) using both restoring and non-restoring division algorithms. Analyze both methods step-by-step, showing intermediate remainders and quotient formation. Compare the two approaches in terms of number of steps, complexity, and efficiency, and explain why non-restoring division is often preferred in modern systems.

5. A scientific application requires precise handling of real numbers using the IEEE 754 standard for floating-point representation. Given a decimal number (e.g., −13.25), analyze how it is represented in both single precision and double precision formats, including sign, exponent, and mantissa fields. Further, explain how floating-point addition is performed between two such numbers, highlighting issues like normalization, rounding, and precision loss.

Code of Conduct:

I B. Poorvaja (192473008) *(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.I understand that any violation of academic integrity rules will result in disciplinary action.*

Signature of the student:

MARKS ALLOCATION

	Problem Understanding (20%)	Method Selection (20%)	Solution Process (25%)	Accuracy of Calculation (20%)	Interpretation of Results (15%)	
Q1	3	4	4	4	2	17
Q2	4	3	3	3	2	15
Q3	3	4	4	4	3	18
Q4	4	3	2	3	2	14
Q5	3	2	2	2	2	11
						75

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Problem Understanding	20%	Clearly interprets problem, identifies all parameters and requirements accurately(4)	Understands problem with minor gaps(3)	Partial understanding; misses key elements(2)	Misinterprets or unable to understand problem(1)
Method Selection	20%	Selects most appropriate method/formula with strong justification(4)	Selects correct method with minor errors(3)	Inappropriate or partially correct method(2)	Unable to select method(1)
Solution Process	25%	Logical, systematic steps with correct approach throughout(5)	Mostly correct steps with minor mistakes(4)	Disorganized steps; multiple errors(3)	No clear process(0)
Accuracy of Calculation	20%	All calculations accurate; correct final answer(4)	Minor calculation errors(3)	Frequent errors; incorrect final answer(2)	No valid calculations(0)
Interpretation of Results	15%	Clearly interprets results with units and engineering relevance(3)	Basic interpretation with minor omissions(2)	Limited or unclear interpretation(1)	No interpretation(0)

SIMATS ENGINEERING
Department of Computer Science & Engineering
ASSESSMENT TOOL 1- ANALYTICAL PROBLEM SOLVING

Course Code:	**CSA1223**	**Course Title:**	**Computer Architecture**
CO Assessed:	**CO2**	**Assessment:**	**ASSESSMENT TOOL1- ANALYTICAL PROBLEM SOLVING**
Weightage:	**45%**	**Total Marks:**	**100**
CO2 Statement:	Apply integer and floating-point arithmetic algorithms including Booth's multiplication, restoring/non-restoring division, and IEEE 754 standard operations to solve fixed-point and floating-point computational problems. (BL3)		
Student Name:	**B. Poorvaja**	**Reg.No.:**	**192473008**
Department:	**B.E-CSE(DS)**	**Date:**	

ANNEXURE A

1. A processor performs arithmetic operations on signed 8-bit integers using 2's complement representation. Consider two numbers: +45 and −23. Analyze in detail how the system performs both addition and subtraction, including binary conversion, alignment of sign bits, and intermediate steps. Determine whether overflow occurs, and explain how the processor detects overflow and interprets the final result.

   Signed 8-bit Arithmetic Using 2's Complement

   The processor uses 8-bit 2's complement representation. The two numbers are +45 and −23. We perform both addition and subtraction.

   Step 1: Convert +45 to binary

   45 in decimal: 45=32+8+4+1

   Therefore, **+45=00101101**

   Step 2: Convert −23 to 2's complement

   First convert +23: **23=00010111**

   Invert all bits: 11101000

   Add 1: 1101000 + 1= 11101001

   Therefore, **−23=11101001**

   A. Addition: +45 + (−23)

00101101

+11101001

Perform binary addition:

  00101101   (+45)

 + 11101001   (-23)

 ----

 1 00010110

Discard the carry beyond 8 bits: 00010110

Convert to decimal: 00010110 =16+4+2=22

Therefore,

45 + (-23) = 22

Overflow check

For signed addition, overflow occurs only when two numbers having the same sign are added and the result has the opposite sign.

Here:

- +45 is positive
- −23 is negative
- Result = +22

Since the operands have different signs, **overflow cannot occur**.

B. Subtraction: +45 − (−23)

Subtraction is performed as addition of the 2's complement: 45−(−23)=45+23

The 2's complement of −23 is +23:

+45 = 00101101

+23 = 00010111

Add:

  00101101

 + 00010111

01000100

01000100=64+4=68

Therefore, **45 – (-23) = 68**

Overflow check

Both operands effectively being added are positive:

00101101 (+45)

+ 00010111 (+23)

01000100 (+68)

The result is still positive and lies within the 8-bit signed range:$-128 \leq x \leq 127$

Therefore:

No overflow

Final interpretation

Operation	Binary Result	Decimal Result	Overflow
+45 + (−23)	00010110	+22	No
+45 − (−23)	01000100	+68	No

The processor detects signed overflow by examining the signs of the operands and result, or equivalently through the relationship between the carry into and carry out of the sign bit.

2. A high-performance computing system replaces a ripple carry adder with a 4-bit carry look-ahead adder (CLA) to reduce computation delay. Given two binary inputs, analyze how generate and propagate signals are formed and how carries are computed in advance. Compare the time delay of CLA with ripple carry addition and justify, with reasoning, why CLA improves speed in modern processors.

4-bit Carry Look-Ahead Adder

The question asks how generate (G) and propagate (P) signals are formed and how carries are calculated in advance. The supplied question does not specify actual binary inputs, so the following demonstrates the operation using:

A=1011, B=0110

Let:

A3 A2 A1 A0 = 1011
B3 B2 B1 B0 = 0110

For each bit, $P_i = A_i \oplus B_i$

Step 1: Calculate P and G

Bit	A	B	P = A⊕B	G = AB
0	1	0	1	0
1	1	1	0	1
2	0	1	1	0
3	1	0	1	0

Therefore: P3 P2 P1 P0 = 1011

G3 G2 G1 G0 = 0100

Assume: $C_0 = 0$

Step 2: Calculate carries in advance

For a CLA:

$C_1 = G_0 + P_0C_0$

$C_2 = G_1 + P_1G_0 + P_1P_0C_0$

$C_3 = G_2 + P_2G_1 + P_2P_1G_0 + P_2P_1P_0C_0$

$C_4 = G_3 + P_3G_2 + P_3P_2G_1 + P_3P_2P_1G_0 + P_3P_2P_1P_0C_0$

Substituting:

C1=0+1(0)=0

C2=1+0=1

C3=0+1(1)=1

C4=0+1(0)+1(1)(1) = 1

Step 3: Calculate sum bits

Si=Pi⊕Ci

Therefore:

S0 = 1 ⊕ 0 = 1
S1 = 0 ⊕ 0 = 0

S2 = 1 ⊕ 1 = 0
S3 = 1 ⊕ 1 = 0

Hence:

A = 1011
B = 0110

Result = 10001

So: 1011+ 0110=10001

CLA vs Ripple Carry Adder

In a Ripple Carry Adder**,** each carry must wait for the previous carry:

C0 → FA0 → C1 → FA1 → C2 → FA2 → C3 → FA3

Thus, the delay increases as the number of bits increases.

In a Carry Look-Ahead Adder, the generate and propagate signals allow carries to be calculated simultaneously:

P/G signals
↓
Carry Look-Ahead Logic
↓
C1 C2 C3 C4
↓
Sum

Therefore, CLA significantly reduces carry propagation delay and is faster for high-performance processors.

3**.** In a signed multiplication operation, a processor uses Booth's algorithm to multiply two numbers, −6 and +5. Analyze the algorithm step-by-step, including the role of the accumulator, multiplier, and Booth bit, along with arithmetic shifts. Explain how the algorithm efficiently handles signed numbers and reduces the number of addition/subtraction operations compared to traditional multiplication.

Booth's Algorithm: −6 × +5

The question specifically asks for Booth's algorithm, including the accumulator, multiplier, Booth bit and arithmetic shifts.

We use 4-bit numbers**.**

Step 1: Represent the numbers −6

+6: 0110

Invert:1001

Add 1:1010

Therefore: M=−6=1010

Multiplier: Q=+5 = 0101

Initial accumulator: A=0000

Booth bit: Q−1=0

−M=+6 = 0110

Booth rules

Q_0	Q_{-1}	**Operation**
0	0	No operation
0	1	A = A + M
1	0	A = A − M
1	1	No operation

After every operation, perform an arithmetic right shift on: A,Q,Q−1

Iteration 1

Initial:

A = 0000 , Q = 0101

Q-1 = 0

Q_0Q_{-1} = 10

Therefore: A=A-M

Since : M= -6:

A=0−(−6) = +6

A = 0110

Arithmetic right shift:

Before: 0110 0101 0

After: 0011 0010 1

So: A = 0011, Q = 0010

Q-1 = 1

Iteration 2

Q_0Q_{-1}=01

Therefore: A=A+M

0011

+ 1010

1101

Thus: A = 1101

Arithmetic right shift:

Before: 1101 0010 1

After: 1110 1001 0

RESULT A = 1110 , Q = 1001

Q-1 = 0

Iteration 3

Q0Q−1=10

Therefore: A=A−M

So: A = 0100

Arithmetic right shift:

Before: 0100 1001 0

After: 0010 0100 1

Therefore: A = 0010, Q = 0100

Q-1 = 1

Iteration 4

Q0Q−1=01

Therefore: A=A+M

0010

+ 1010

1100

Arithmetic right shift:

Before: 1100 0100 1

After: 1110 0010 0

Final: A = 1110 , Q = 0010

Therefore the product is: **AQ = 11100010**

Interpret 11100010 as an 8-bit 2's-complement number.

Invert: 00011101

Add 1: **00011110**

00011110=30

Therefore: -6 x 5 = -30

Why Booth's algorithm is efficient

Booth's algorithm examines adjacent multiplier bits and converts consecutive 1s into fewer arithmetic operations. Instead of performing an addition for every 1 in the multiplier, it uses add/subtract operations followed by arithmetic shifts.

It is particularly useful because it:

- Handles positive and negative numbers naturally.
- Uses 2's-complement representation.
- Reduces the number of addition/subtraction operations when there are consecutive 1s.
- Supports signed multiplication efficiently.

4. A processor is required to perform division of two binary numbers (e.g., 13 ÷ 3) using both restoring and non-restoring division algorithms. Analyze both methods step-by-step, showing intermediate remainders and quotient formation. Compare the two approaches in terms of number of steps, complexity, and efficiency, and explain why non-restoring division is often preferred in modern systems.

Restoring and Non-Restoring Division: 13 ÷ 3

The question asks to perform both methods and compare their efficiency.

We use: 13=110, 3=0011

Therefore, the expected result is:

13÷3= 4 remainder 1

A. Restoring Division

Initial values:

Dividend Q = 1101 , Divisor M = 0011

A = 0000

For each iteration:

1. Shift A and Q left.
2. Subtract M from A.
3. If A becomes negative, restore A and put 0 in Q_0.
4. If A remains positive/non-negative, put 1 in Q_0.

Iteration 1

Shift: A = 0001 , Q = 1010

Subtract: 0001 - 0011 = -0010

Negative, so restore: A = 0001, Q0 = 0

Iteration 2

Shift: A = 0011, Q = 0100

Subtract: 0011 - 0011 = 0000

Non-negative:A = 0000, Q0 = 1

Thus: Q = 0101

Iteration 3

Shift: A = 0000, Q = 1010

Subtract:0000 - 0011 = -0011

Negative → restore: A = 0000 , Q0 = 0

Iteration 4

Shift: A = 0001, Q = 0100

Subtract: 0001 - 0011 = -0010

Negative → restore: A = 0001, Q0 = 0

Final:

Q = 0100, A = 0001

Therefore:

Q=0100=4

R=0001=1

Hence: 13 divided by 3 = 4 remainder 1

B. Non-Restoring Division

Non-restoring division avoids immediately restoring the accumulator whenever the subtraction gives a negative result.

Initial:

A = 0000 , Q = 1101 , M = 0011

Iteration 1

Shift A,Q: A = 0001, Q = 1010

A is positive, so subtract M: 0001−0011= -2

A = -2, Q0 = 0

Iteration 2

Since A is negative, add M after shifting.

After shift: A = -4, Q = 0100

Add M:

−4+3= -1

A = -1 , Q0 = 0

Iteration 3

A is negative, so after shifting and adding M:

A = -3 , Q = 1010

Then: −3+3= 0

A = 0000, Q0 = 1

Thus: Q = 1011

Iteration 4

A is non-negative, so subtract M: A−M

After the shift and subtraction: A = -2, Q = 0100

Since A is negative: Q0 = 0

At the end, because A is negative, perform the final correction: A = A + M

−2+3=1

Therefore: A = 0001, Q = 0100

Hence: Q = 4, R= 1

So: 13 divided by 3 = 4 remainder 1

Restoring vs Non-Restoring Division

Feature	**Restoring**	**Non-Restoring**
Negative remainder	Immediately restored	Not immediately restored
Addition/subtraction	More operations	Fewer operations
Hardware complexity	Relatively simple	Slightly more complex control
Speed	Lower	Higher
Efficiency	Less efficient	More efficient
Final correction	Not generally required	Required if final remainder is negative

Conclusion

Both algorithms produce: **13 divided by 3 = 4 remainder 1**

Non-restoring division is generally preferred for high-performance implementations because it avoids the repeated subtract-then-restore operation, reducing the number of arithmetic operations and improving efficiency.

5.A scientific application requires precise handling of real numbers using the IEEE 754 standard for floating-point representation. Given a decimal number (e.g., −13.25), analyze how it is represented in both single precision and double precision formats, including sign, exponent, and mantissa fields. Further, explain how floating-point addition is performed between two such numbers, highlighting issues like normalization, rounding, and precision loss.

IEEE 754 Representation of −13.25

The question requires both **single precision and double precision**, including sign, exponent, mantissa, and floating-point addition concepts.

The number is: -13.25

Step 1: Convert 13.25 to binary

Integer part: $13=1101_2$

Fractional part: $0.25=0.01_2$

Therefore: $13.25=1101.01_2$

Since the number is negative: $-13.25=-1101.01_2$

Step 2: Normalize

Move the binary point until there is one digit before it: $1101.01=1.10101\times2^3$

Therefore: $-13.25=-1.10101\times2^3$

A. IEEE 754 Single Precision

Single precision uses:

- 1 sign bit
- 8 exponent bits
- 23 fraction bits

Sign : The number is negative: S=1

Exponent

Actual exponent: 3

Single precision bias: 127

Therefore: 3+127=130

130 in binary: $130=10000010_2$

Therefore: 10000010

Mantissa/Fraction

From: 1.10101×2^3

The leading 1 is implicit.

Fraction: 10101

Pad to 23 bits: 10101000000000000000000

Therefore:

Sign Exponent Fraction

1 10000010 10101000000000000000000

Complete 32-bit representation: 1 10000010 10101000000000000000000

or: 11000001010101000000000000000000

Thus: -13.25 = 11000001010101000000000000000000 in IEEE 754 single precision.

B. IEEE 754 Double Precision

Double precision uses:

- 1 sign bit
- 11 exponent bits
- 52 fraction bits

Sign , S = 1

Exponent

Actual exponent: 3

Double precision bias: 1023

Therefore: 3+1023=1026

1026 in binary: 10000000010

Therefore: 10000000010

Fraction

Normalized number: 1.10101×2^3

Fraction: 10101

Pad to 52 bits: 1010100000000000000000000000000000000000000000000000

Therefore:

Sign | Exponent | Fraction

1 | 10000000010 | 1010100000000000000000000000000000000000000000000000

Complete representation:

1 10000000010 1010100000000000000000000000000000000000000000000000

Since −13.25 is exactly representable in binary, **no rounding is required** for either single or double precision

Floating-Point Addition

When adding two IEEE 754 floating-point numbers, the processor generally follows these steps:

1. Compare exponents

The exponents of the two numbers are compared.

2. Align mantissas

The mantissa of the number with the smaller exponent is shifted right until both exponents match.

Example: $1.101\times2^3+1.010\times2^1$

The second number is shifted: $.010\times2^1=0.01010\times2^3$

Now both numbers have exponent 3.

3. Add or subtract mantissas

1.10100

+ 0.01010

1.11110

4. Normalize

The result is adjusted to the form: $1.xxxxx\times2^n$

If the result is: 10.111×2^3

it is normalized to: 1.0111×2^4

5. Round

If the result contains more bits than the available fraction field, it is rounded according to the IEEE 754 rounding mode, commonly **round-to-nearest, ties-to-even**.

6. Store the result

The processor stores: Sign | Biased Exponent | Fraction

Precision Loss

Floating-point precision can be lost mainly because the fraction field has a finite number of bits.

During addition:

- Exponents may need alignment.
- Right shifting can discard low-order bits.
- The final mantissa may need rounding.
- Very large and very small numbers added together may result in the smaller number having little or no effect.

For example, when two numbers have very different exponents, aligning them can shift the smaller number's significant bits beyond the available precision.

Code of Conduct:

I _B.Poorvaja(192473008)_ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

MARKS ALLOCATION

	Problem Understanding (20%)	**Method Selection (20%)**	**Solution Process (25%)**	**Accuracy of Calculation (20%)**	**Interpretation of Results (15%)**
Q1					
Q2					
Q3					
Q4					
Q5					

RUBRICS FOR CALCULATION:

Criteria	**Weightage**	**Level 4 – Exemplary**	**Level 3 – Proficient**	**Level 2 – Developing**	**Level 1 – Beginning**
Problem Understanding	20%	Clearly interprets problem, identifies all parameters and requirements accurately(4)	Understands problem with minor gaps(3)	Partial understanding; misses key elements(2)	Misinterprets or unable to understand problem(1)
Method Selection	20%	Selects most appropriate method/formula with strong justification(4)	Selects correct method with minor errors(3)	Inappropriate or partially correct method(2)	Unable to select method(1)
Solution Process	25%	Logical, systematic steps with correct approach throughout(5)	Mostly correct steps with minor mistakes(4)	Disorganized steps; multiple errors(3)	No clear process(0)
Accuracy of Calculation	20%	All calculations accurate; correct final answer(4)	Minor calculation errors(3)	Frequent errors; incorrect final answer(2)	No valid calculations(0)
Interpretation of Results	15%	Clearly interprets results with units and engineering relevance(3)	Basic interpretation with minor omissions(2)	Limited or unclear interpretation(1)	No interpretation(0)